

Sistema de Control de Laboratorios e Infraestructura (SCLI) Entrega 4

Equipo FUVV

Repositorio: <https://github.com/ffarinangog2/Entrega-final-del-PFC>

Rama de trabajo: feature/entrega-4

Índice

1. Introducción	3
2. Arquitectura de la solución	3
2.1. Límites de servicio y comunicación	3
2.2. Decisiones arquitectónicas registradas	4
3. Arquitectura por capas y principios SOLID	4
3.1. Organización del código	4
3.2. Aplicación razonada de SOLID	5
4. Patrones de diseño aplicados	5
4.1. Repository y Factory Method	5
4.2. Strategy y State	5
4.3. Observer, Facade y Singleton	6
5. Aplicaciones e integración funcional	6
5.1. Backend distribuido	6
5.2. Autenticación y gestión de sesión	7
5.3. Aplicación web	8
5.4. Flujo web de autenticación	8
5.5. Aplicación móvil Android	9
5.5.1. Persistencia local y funcionamiento sin conexión	9
5.5.2. QR y capacidades del dispositivo	9
5.5.3. Incidentes y notificaciones	10
5.6. Integración de clientes y servicios	10
5.7. Seguridad funcional y limitaciones	10
6. Estrategia de pruebas y entrega continua	11
6.1. Backend: pruebas unitarias, integración y cobertura	11
6.1.1. Persistencia real con Testcontainers	12
6.2. Aplicación web	12

6.2.1.	Compatibilidad E2E con Playwright	13
6.3.	Aplicación móvil	13
6.3.1.	Pruebas instrumentadas y APK	14
6.4.	Contratos HTTP con Pact	14
6.5.	Carga nominal y rampa	14
6.6.	Integración y entrega continuas	15
6.6.1.	Artefactos y trazabilidad por SHA	15
6.6.2.	Despliegue controlado	15
6.7.	Trazabilidad de requisitos de calidad	16
7.	Observabilidad y evaluación ISO/IEC 25010	16
7.1.	Arquitectura de observabilidad	17
7.1.1.	Métricas Prometheus	17
7.1.2.	Trazas y logs estructurados	17
7.2.	Dashboard consolidado	18
7.3.	Marco y protocolo ISO/IEC 25010	18
7.3.1.	Proveniencia e integridad	19
7.4.	Eficiencia nominal: 50 usuarios durante cinco minutos	19
7.4.1.	Contraste con Prometheus	20
7.5.	Rampa exploratoria de 0 a 200 usuarios	20
7.6.	Fiabilidad y disponibilidad: alcance no concluido	20
7.7.	Experimento de arbitraje concurrente S0–S4	20
7.8.	Evaluación ISO/IEC 25010 seleccionada	22
7.9.	Mantenibilidad: evidencia y alcance	22
7.10.	Análisis crítico	23
7.11.	Reflexión ética	24
7.12.	Declaración de uso de inteligencia artificial generativa	24
7.13.	Amenazas a la validez	25
7.13.1.	Validez interna	25
7.13.2.	Validez externa	25
7.13.3.	Validez de constructo	25
7.14.	Trazabilidad final de la Entrega 4	25
7.15.	Conclusiones	25
7.16.	Trabajo futuro	27

1. Introducción

La Entrega 4 consolida el sistema distribuido de gestión de laboratorios como un producto integrado. El trabajo parte de la infraestructura distribuida ya disponible, pero su aportación no consiste en presentar nuevamente esa base: se centra en delimitar los servicios, ordenar el código de negocio, ofrecer clientes web y móvil y establecer mecanismos verificables de seguridad, pruebas, integración y despliegue continuos y observabilidad. La arquitectura se trata, por tanto, como un conjunto de decisiones que condicionan atributos de calidad y no solamente como un diagrama de componentes. Esta perspectiva es coherente con el papel que la literatura asigna a la arquitectura en la modificabilidad, la seguridad, el rendimiento y la testabilidad de un sistema [1].

La solución incorpora una aplicación web en React y una aplicación Android con Jetpack Compose, ambas consumidoras de un punto de entrada común. La seguridad se aplica mediante autenticación JWT, sesiones de renovación y autorización por roles; su centralización en el perímetro reduce exposiciones innecesarias, aunque no sustituye los controles de cada servicio. Esta defensa en varios niveles es relevante porque la comunicación distribuida amplía la superficie de ataque y obliga a considerar conjuntamente identidad, infraestructura y desarrollo [2].

La verificación comprende pruebas unitarias, de integración, contratos, interfaces y carga, coordinadas por el flujo de CI/CD. Prometheus, Grafana, OpenTelemetry, Tempo, Loki y cAdvisor aportan métricas, trazas, registros y datos de recursos para observar el comportamiento desplegado. Finalmente, la evaluación adopta el modelo de calidad de producto ISO/IEC 25010:2023, que proporciona características y subcaracterísticas para especificar y evaluar la calidad de productos TIC [3]. Los capítulos posteriores distinguen las propiedades demostradas de las mediciones incompletas, evitando convertir configuraciones o pruebas no ejecutadas en resultados experimentales.

2. Arquitectura de la solución

2.1. Límites de servicio y comunicación

El sistema adopta una descomposición por capacidades, formalizada en el ADR-002¹. `auth-service` gestiona autenticación, emisión de tokens y sesiones de renovación; `usuarios-service` administra perfiles institucionales; `academico-laboratorios-service` mantiene los catálogos académicos y el inventario de espacios y equipos; y `reservas-solicitudes-servicio` controla solicitudes, disponibilidad, bloqueos y reservas. El `api-gateway` completa la topología como punto de entrada para ruteo, CORS y validación de autenticación, sin incorporar reglas del dominio.

Los clientes React y Android acceden a las API a través del Gateway. Las operaciones que requieren información de otra capacidad utilizan contratos HTTP explícitos entre servicios; no comparten repositorios ni entidades JPA. Cada servicio conserva sus migraciones y su base lógica en CockroachDB. Esta separación favorece la evolución y prueba independiente, pero introduce latencia, errores parciales y compatibilidad de contratos como preocupaciones de diseño. La literatura reporta precisamente que el despliegue y los patrones de comunicación son tan relevantes como la delimitación inicial de los microservicios [4].

¹Evidencia interna: `../adr/ADR-002-microservicios.md`.

Docker proporciona unidades reproducibles de ejecución para los servicios y la infraestructura de soporte. El flujo observable es: cliente, Gateway, servicio propietario del caso de uso y persistencia. En paralelo, los servicios exportan telemetría hacia la infraestructura de observabilidad. Los diagramas C4 de contexto y contenedores existentes² documentan estos límites; se referencian sin duplicarlos dentro del nuevo informe.

La descomposición no elimina los compromisos propios de un sistema distribuido. No existe una transacción ACID única que abarque las bases de todos los servicios, de modo que los casos de uso interservicio deben gestionar fallos y consistencia de forma explícita. Asimismo, el Gateway constituye un punto común de acceso, pero los servicios siguen siendo responsables de validar las operaciones que protegen. Estas consecuencias se consideran parte de la arquitectura real y no defectos ocultos por la representación C4.

2.2. Decisiones arquitectónicas registradas

Los ADR preservan el contexto y las consecuencias de decisiones que el código por sí solo no explica. El ADR-001³ registra la reorganización por capas de `academico-laboratorios-service`; el ADR-002 fija los límites de los microservicios y la propiedad de sus datos. El ADR-005⁴ identifica los patrones que participan en rutas reales y también documenta deuda arquitectónica. Por último, el ADR-006⁵ justifica Kotlin y Jetpack Compose, MVVM, Room y las integraciones nativas como base del cliente móvil.

Los ADR-003 y ADR-004 no se presentan como resultados de esta entrega: sus decisiones sobre distribución de datos constituyen antecedentes de la infraestructura. Esta distinción evita atribuir a la Entrega 4 trabajo ya documentado y mantiene el foco en la consolidación arquitectónica y de calidad.

3. Arquitectura por capas y principios SOLID

3.1. Organización del código

Los servicios con reglas de negocio se organizan, con variaciones heredadas, en cuatro zonas. La capa *presentation* contiene controladores REST, DTO y la traducción de excepciones a respuestas HTTP. La capa *application* orquesta servicios y casos de uso; decide el orden de las operaciones, pero delega las reglas específicas. *Domain* contiene modelos, estados, estrategias, eventos y puertos que expresan las necesidades del negocio. Finalmente, *infrastructure* aporta adaptadores HTTP, seguridad, observabilidad y persistencia, incluidas las entidades JPA y las implementaciones Spring Data.

Esta dirección permite que un controlador dependa de un caso de uso y que el caso de uso dependa de un puerto, mientras el adaptador concreto implementa ese puerto desde el exterior. En consecuencia, una prueba puede reemplazar la base de datos o un cliente remoto por un doble sin trasladar detalles de JPA al escenario de negocio. El API Gateway es una excepción deliberada: sus paquetes de configuración, rutas, seguridad y registro corresponden a infraestructura; crear para él un dominio vacío no mejoraría la separación.

²Evidencia interna: `../diagrams/c4-nivel1.png` y `../diagrams/c4-nivel2.png`.

³Evidencia interna: `../adr/ADR-001-arquitectura.md`.

⁴Evidencia interna: `../adr/ADR-005-patrones-gof.md`.

⁵Evidencia interna: `../adr/ADR-006-eleccion-movil.md`.

3.2. Aplicación razonada de SOLID

La responsabilidad única se refleja en controladores que adaptan HTTP, servicios que coordinan casos de uso y repositorios que resuelven persistencia. La inversión de dependencias se materializa cuando aplicación y dominio conocen interfaces como `UsuarioRepository` o los puertos `*RepositoryPort`, y los adaptadores de infraestructura dependen de esas abstracciones. Strategy y State ofrecen puntos de extensión sin reescribir los servicios coordinadores, lo que aproxima el principio abierto/cerrado. Las interfaces son acotadas al comportamiento requerido por sus consumidores, y las implementaciones de estados pueden sustituirse a través de sus contratos.

Estos elementos mejoran mantenibilidad y testabilidad, pero no demuestran un cumplimiento perfecto de SOLID. En `academico-laboratorios-service`, varios puertos del dominio importan `org.springframework.data.domain.Page` y `org.springframework.data.domain.Pa`. El dominio queda así acoplado al modelo de paginación de Spring Data, pese a que la persistencia se implemente mediante adaptadores. Sustituir esos tipos por contratos propios y mapearlos en infraestructura cerraría la brecha, pero exige una migración coordinada de servicios y controladores. Se registra, por ello, como deuda técnica y no se oculta bajo una afirmación absoluta de independencia.

4. Patrones de diseño aplicados

Los patrones siguientes se identificaron por sus participantes, consumidores y comportamiento, no solamente por el nombre de una clase. Su uso responde a problemas concretos del sistema y complementa la separación por capas.

4.1. Repository y Factory Method

Repository. El problema consiste en evitar que los casos de uso conozcan JPA o Spring Data. En Auth, `UsuarioRepository` define el puerto del dominio y `UsuarioPersistenceAdapter` lo implementa mediante `UsuarioAuthRepository`; el mismo esquema puerto–adaptador aparece en Usuarios, Académico y Reservas. El beneficio es sustituir la persistencia en pruebas y concentrar el mapeo. Su aplicación es parcial en los puertos paginados de Académico por la dependencia de Spring ya señalada.

Factory Method. La construcción del principal de seguridad no debe quedar fijada en el servicio que carga usuarios. `UserDetailsFactory` declara `crear(Usuario)` y `DefaultUserDetailsFactory` produce `CustomUserDetails`; `CustomUserDetailsService` consume la interfaz. La construcción queda encapsulada y puede sustituirse en pruebas sin duplicar conocimiento del producto concreto.

4.2. Strategy y State

Strategy. La disponibilidad de un laboratorio es una política que puede evolucionar sin reescribir la orquestación. La interfaz `DisponibilidadStrategy` define la evaluación y `DisponibilidadSinConflictosStrategy` comprueba fecha, laboratorio, cruces y bloqueos. `StrategyConfig` selecciona la implementación y `DisponibilidadServiceImpl` la utiliza. Aunque actualmente existe una sola estrategia concreta, el punto de variación es real, puro y probado.

State. Las transiciones de solicitudes y reservas no se distribuyen en condicionales de los controladores. `SolicitudReservaState` y `ReservaState` definen los contratos; sus estados concretos encapsulan las acciones permitidas, mientras `SolicitudReservaServiceImpl` y `ReservaServiceImpl` delegan en ellos. El patrón centraliza invariantes y facilita probar transiciones, a cambio de mantener más clases y actualizar el selector cuando se incorpora un estado.

4.3. Observer, Facade y Singleton

Observer. La creación o el cambio de estado de un perfil debe poder producir efectos secundarios sin acoplarlos al caso de uso. `PerfilEventListener` define el observador, `LoggingPerfilEventListener` implementa el registro y `PerfilServiceImpl` publica `PerfilEvent`. La extensión con otros observadores no modifica el servicio; como consecuencia crítica, la notificación actual es síncrona y un listener costoso podría aumentar la latencia.

Facade. Construir el detalle completo de un laboratorio requiere coordinar laboratorio, piso, bloque, campus y equipos. `LaboratorioDetalleFacadeImpl`, a través de `LaboratorioDetalleFacade`, ofrece una operación única y evita que el controlador replique esa coordinación. El beneficio es una interfaz simple para una composición real; la implementación mantiene, no obstante, una dependencia con un DTO de presentación que constituye otra oportunidad de desacoplamiento.

Singleton. El conteo de peticiones HTTP requiere un registro único por aplicación para no fragmentar las métricas. Las implementaciones `HttpRequestsMetricsRegistry` de Académico y Usuarios son componentes de Spring, conservan una instancia compartida y la exponen mediante `getInstance()`; los filtros HTTP son sus clientes. Se obtiene un punto de registro coherente, pero se mezcla el ciclo de vida del contenedor con estado estático, lo que dificulta su sustitución frente a una inyección de constructor convencional.

En conjunto, los siete patrones resuelven persistencia, creación, variación de políticas, transiciones, reacción a eventos, composición y registro compartido. Su valor no reside en maximizar el número de patrones, sino en hacer explícitas decisiones que reducen acoplamiento en rutas verificables. Las limitaciones descritas muestran que el diseño sigue siendo perfectible y proporcionan una base concreta para trabajo posterior.

5. Aplicaciones e integración funcional

La Entrega 4 incorpora dos clientes sobre un backend distribuido común. La aplicación web atiende la operación institucional desde navegador y la aplicación móvil facilita tareas de consulta y registro desde Android. Ambas reutilizan los mismos contratos HTTP y atraviesan el API Gateway; por ello, la autorización y las reglas de negocio no se duplican en cada interfaz. Esta organización concreta los límites descritos en la Sección 2 y conserva la independencia entre presentación, servicios y persistencia.

5.1. Backend distribuido

El backend se compone de cinco aplicaciones Spring que cooperan mediante HTTP. La separación permite evolucionar capacidades de autenticación, usuarios, catálogos y reservas de manera acotada, aunque exige gestionar errores parciales y contratos entre procesos, una consecuencia característica de la comunicación entre microservicios [4].

auth-service autentica credenciales, emite y renueva JWT, mantiene sesiones de renovación y ejecuta recuperación de contraseña. Para construir la identidad presentada a los clientes consulta el perfil institucional en **usuarios-service**. Este último administra perfiles, estudiantes, docentes y administradores; expone operaciones de consulta, creación, actualización y cambio de estado, y persiste sus agregados en CockroachDB. Los cambios de perfil producen eventos consumidos actualmente por un observador de registro, aplicación del patrón ya analizado en la Sección 4.

academico-laboratorios-service gestiona campus, bloques, pisos, laboratorios, equipos y tipos de equipo, además de facultades, carreras, materias, periodos lectivos y horarios académicos. Sus API permiten mantener los catálogos, consultar laboratorios disponibles y obtener información individual. La operación de detalle completo coordina laboratorio, ubicación y equipos mediante **LaboratorioDetalleFacade**; este es el contrato que utiliza el flujo QR móvil después de identificar un laboratorio.

reservas-solicitudes-service concentra solicitudes, reservas, disponibilidad y agenda. Permite crear y actualizar solicitudes, consultar listados, detalle e historial, poner una solicitud en revisión, aprobarla o rechazarla, cancelarla y proponer o responder una alternativa de horario y laboratorio. Para las reservas ofrece listado, detalle, calendario y transiciones de cancelación, inicio, finalización y no asistencia. Strategy encapsula la comprobación de disponibilidad y State controla las transiciones, sin trasladar esas reglas a los clientes. La creación y la aprobación emplean **Idempotency-Key** y registros persistentes en CockroachDB para impedir que una repetición equivalente produzca resultados duplicados.

Finalmente, **api-gateway** es el punto de entrada de Web y Mobile. Encamina las familias **/api/v1/auth**, **/api/v1/perfiles**, **/api/v1/laboratorios**, **/api/v1/solicitudes** y **/api/v1/reservas** hacia su servicio propietario. Valida firma, emisor, vigencia y tipo *access* del JWT antes de admitir una ruta protegida. Los servicios vuelven a aplicar permisos específicos, de modo que una identidad ausente o inválida genera 401 y una identidad autenticada sin autoridad genera 403. El Gateway también fija cabeceras de seguridad, integra telemetría y aplica CORS con una lista de orígenes configurable mediante entorno, no con una IP de despliegue incorporada al código. La concentración de controles en el perímetro reduce inconsistencias, pero no reemplaza la seguridad interservicio, como advierten las guías de seguridad para aplicaciones de microservicios [5].

5.2. Autenticación y gestión de sesión

El inicio de sesión admite nombre de usuario o correo institucional y verifica la contraseña con BCrypt, configurado con factor de coste 12. Tras autenticar, **auth-service** recupera el perfil y construye una respuesta con roles, permisos, token de acceso de corta duración y token de renovación. Las sesiones de renovación se guardan en CockroachDB mediante el hash SHA-256 del token; no se persiste su valor en claro. Una renovación valida que la sesión continúe activa, revoca el token anterior y crea otro dentro de la misma familia. La rotación reduce la ventana de reutilización de un refresh token, práctica recomendada para clientes públicos [6].

La protección de acceso registra intentos y aplica un bloqueo temporal de quince minutos al alcanzar cinco fallos consecutivos. Un acceso correcto limpia el contador. La recuperación de contraseña devuelve un mensaje neutral para no revelar si el identificador existe, limita solicitudes y utiliza tokens de restablecimiento con expiración e invalidación. Roles y permisos se incluyen en la identidad autenticada, pero la decisión final se mantiene

en las reglas de Spring Security de cada servicio.

El cierre de sesión web sigue un flujo completo y verificable:

1. el cliente lee el refresh token de la sesión y envía `POST /api/v1/auth/logout`;
2. el Gateway dirige la solicitud a `auth-service`;
3. el servicio valida el token y revoca de forma idempotente la sesión persistida;
4. el cliente elimina access token, refresh token, expiración y usuario de `sessionStorage`, actualiza el estado React y redirige a `/login`.

La limpieza local se realiza incluso si la revocación remota no puede completarse, para no dejar la interfaz atrapada en una sesión aparente. Esto prioriza el cierre en el dispositivo, aunque una interrupción de red impide confirmar la revocación en el servidor y debe considerarse al evaluar el riesgo residual.

5.3. Aplicación web

La aplicación web es una SPA construida con React 18.3.1 y TypeScript 5.8 en modo `strict`. React Router separa las rutas públicas de inicio y recuperación de contraseña de las rutas autenticadas. Dentro del área protegida, `ProtectedRoute` comprueba roles o permisos antes de presentar reservas, solicitudes, calendario y creación de solicitudes. Esta comprobación mejora la experiencia al ocultar acciones no disponibles, pero el backend conserva la autoridad efectiva y responde 403 cuando corresponde.

El contexto de autenticación guarda access token, refresh token, vencimiento y perfil en `sessionStorage`. No se afirma el uso de cookies `httpOnly`: los JWT están disponibles al código del cliente y se eliminan al cerrar la sesión o la pestaña. Tampoco se almacenan credenciales en `localStorage`. Este último se utiliza exclusivamente para preferencias no sensibles de idioma y tema mediante Zustand. Cada solicitud protegida añade el Bearer token; ante un 401, el cliente coordina una sola renovación en curso, actualiza ambos tokens y reintenta la operación. Si la renovación falla, limpia la sesión. Los mensajes distinguen sesión expirada (401) de falta de permisos (403).

En el dominio funcional, la Web presenta laboratorios y métricas de ocupación, gestión administrativa de perfiles, listas de solicitudes y reservas, detalle e historial, calendario semanal, creación y cancelación, revisión, aprobación, rechazo y propuestas alternativas según permisos. Los formularios consultan catálogos y disponibilidad antes de enviar operaciones. Las pantallas modelan estados de carga, vacío y error; en listados críticos se ofrece reintento y los errores HTTP se traducen a mensajes comprensibles.

La internacionalización utiliza `i18next` con recursos en español e inglés, y el tema permite modos claro y oscuro. En desarrollo, Vite redirige las rutas relativas `/api`; el artefacto de producción se sirve mediante Nginx, que mantiene la navegación de la SPA y actúa como proxy hacia el Gateway. Por ello, el navegador no necesita conocer direcciones internas de microservicios.

5.4. Flujo web de autenticación

La Figura 1 resume el recorrido comprobado en el código. No representa un protocolo adicional: sintetiza las llamadas HTTP y los estados descritos anteriormente.

```

Usuario -> Login React -> API Gateway -> auth-service
                                     <- Access + Refresh JWT
-> sessionStorage -> rutas protegidas -> microservicios
-> 401 -> /auth/refresh -> rotacion -> reintento
-> logout -> /auth/logout -> revocacion -> limpieza -> /login

```

Figura 1: Flujo funcional de autenticación de la aplicación web.

5.5. Aplicación móvil Android

El cliente móvil se desarrolló en Kotlin con Jetpack Compose, Navigation Compose y una organización MVVM por funcionalidad. Compila contra Android API 34, define `targetSdk 34` y soporta desde `minSdk 26`. ViewModels exponen estados de carga, éxito y error mediante `StateFlow`; repositorios separan las pantallas de Retrofit, Room y el almacenamiento de sesión. Esta elección sigue prácticas contemporáneas de desarrollo Android con Kotlin y Jetpack [7].

Retrofit y OkHttp consumen el mismo Gateway que la Web. Un interceptor agrega el access token y un `Authenticator` serializa la renovación ante 401, evita bucles de reintentos y reconstruye la solicitud con el token actualizado. `EncryptedAuthStorage` conserva access token, refresh token, expiración, identidad, roles y permisos mediante `EncryptedSharedPreferences`, con claves AES-256 respaldadas por `MasterKey`. La navegación habilita acciones según permisos. A diferencia de la Web, el logout móvil actual elimina la sesión local, pero no invoca el endpoint remoto de revocación; esta diferencia se registra como limitación.

La aplicación permite listar solicitudes y reservas, consultar sus detalles e historial, crear solicitudes rápidas, verificar disponibilidad, cancelar, revisar, aprobar o rechazar y gestionar propuestas cuando el perfil posee los permisos necesarios. También presenta un calendario. Las operaciones de creación y aprobación envían claves de idempotencia al backend. Los fallos de red y las respuestas 401, 403 y otros errores HTTP se convierten en estados diferenciados para la interfaz.

5.5.1. Persistencia local y funcionamiento sin conexión

Room define una base local con dos conjuntos de datos. Los incidentes se guardan en la tabla `incidentes` y se observan como un flujo reactivo; su creación es enteramente local. Las reservas obtenidas remotamente se almacenan en `reservas_cache`. Si el listado o el detalle fallan por conectividad, el repositorio devuelve los datos disponibles en esa caché e informa que la actualización remota no se completó.

Este comportamiento proporciona lectura degradada, pero no constituye una sincronización offline completa. No hay cola persistente de escrituras, `WorkManager` ni resolución de conflictos; crear o modificar reservas requiere conectividad. Asimismo, los incidentes no tienen fuente remota ni proceso de sincronización. La distinción es importante porque una arquitectura *offline-first* completa debe coordinar explícitamente fuentes locales y de red, así como la reconciliación posterior [8].

5.5.2. QR y capacidades del dispositivo

El flujo QR constituye una capacidad implementada. La pantalla solicita en tiempo de ejecución el permiso de cámara, utiliza `CameraX` para previsualización y análisis, y aplica `ML Kit Barcode Scanning` sobre el fotograma más reciente. El valor leído debe

representar un UUID de laboratorio; el ViewModel evita procesar duplicados y, tras validarlo, consulta `/api/v1/laboratorios/{id}/detalle-completo` a través del Gateway. La interfaz diferencia QR inválido, fallo de red y error del servicio. Por tanto, la lectura de cámara y la consulta posterior están conectadas, aunque dependen de que el laboratorio exista y el backend esté accesible.

5.5.3. Incidentes y notificaciones

La aplicación crea y lista incidentes locales y puede producir una notificación del dispositivo al registrar uno. Para Android 13 o superior solicita `POST_NOTIFICATIONS`; en versiones anteriores utiliza el canal de notificación configurado. Esta notificación local no debe confundirse con una comunicación de servidor.

La integración de Firebase Cloud Messaging abarca más que una preparación del cliente. Android recibe mensajes, crea notificaciones y conserva las renovaciones de token hasta registrarlas mediante `/api/v1/notificaciones/dispositivos`. Reservas persiste dispositivos por perfil, dispone de un adaptador Firebase condicionado por `FIREBASE_ENABLED` y emite eventos de solicitud, incidente y planificación. El token no se escribe en logs y los fallos de entrega no revierten la operación de negocio.

No obstante, `google-services.json` no está versionado y el despliegue no aporta en el repositorio credenciales Firebase reales, como corresponde a secretos. Tampoco existe evidencia conservada de la cadena completa evento backend → Firebase → dispositivo con Play Services → notificación recibida. Por ello FCM está **implementado parcialmente y pendiente de validación E2E**; no se declara operativo extremo a extremo. Además, los disparadores encontrados no incluyen todos los eventos posibles del dominio, por ejemplo la apertura de asistencia.

5.6. Integración de clientes y servicios

La Figura 2 muestra la integración compartida. Web y Mobile mantienen presentaciones y almacenamiento propios, pero reutilizan las mismas API, el esquema Bearer y las decisiones de autorización del backend.

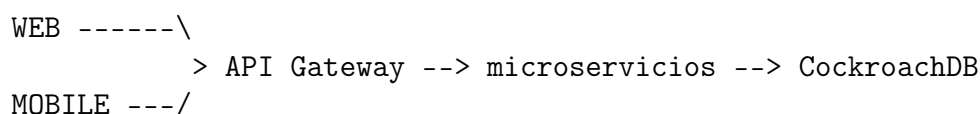


Figura 2: Integración conceptual de clientes, Gateway y servicios.

Los contratos HTTP impiden que los clientes accedan directamente a bases de datos o dependan de entidades de persistencia. El JWT transporta identidad y autoridades; el Gateway valida el token y cada servicio aplica permisos sobre su dominio. Las consultas internas de Auth y Reservas hacia Usuarios o Académico utilizan una cabecera de API interna configurable por entorno. No se reproducen sus valores en el informe ni se incorporan secretos al cliente.

5.7. Seguridad funcional y limitaciones

La seguridad funcional combina contraseñas BCrypt, JWT de acceso, refresh tokens rotados, sesiones persistidas y revocables, RBAC, respuestas diferenciadas 401 y 403,

CORS configurable, cabeceras de seguridad y claves para llamadas internas. Web conserva credenciales exclusivamente en `sessionStorage`; Mobile las cifra mediante Android Security Crypto. Ningún cliente incluye tokens en URL y el servicio FCM no registra el valor recibido. Estos controles reducen riesgos, pero no justifican afirmar seguridad absoluta; la literatura identifica la seguridad de microservicios como un problema transversal de identidad, infraestructura, comunicación y ciclo de desarrollo [2].

Permanecen cuatro limitaciones funcionales. Primero, incidentes es un módulo local sin API ni sincronización remota. Segundo, FCM carece de configuración Firebase verificable, registro de dispositivo y emisor backend. Tercero, salvo la lectura de reservas almacenadas y las operaciones locales de incidentes, las funciones remotas dependen de conectividad y disponibilidad del Gateway. Cuarto, el logout móvil no revoca todavía la sesión de renovación en el backend. La calidad y cobertura de estas funciones se evalúan en el capítulo de pruebas; no se infiere su suficiencia a partir de la mera existencia de pantallas o clases.

No se incorporaron capturas: el repositorio no contiene imágenes actuales y claramente atribuibles a las interfaces E4. Esta omisión evita reutilizar como evidencia visual las capturas históricas de infraestructura.

6. Estrategia de pruebas y entrega continua

La calidad de las aplicaciones descritas en la Sección 5 se evaluó mediante niveles complementarios. Las pruebas unitarias aíslan reglas y estados; las de integración ejercitan frameworks, persistencia y procesos; Pact comprueba contratos entre consumidores y proveedores; Playwright y las pruebas instrumentadas recorren interfaces completas; y Locust genera tráfico concurrente contra el sistema desplegado. Esta separación evita atribuir a una sola métrica riesgos que pertenecen a niveles distintos y sigue el principio de definir procesos de prueba adecuados al contexto del ciclo de vida [9].

La cobertura se interpreta por componente. No se promedian porcentajes de Java, TypeScript y Kotlin, porque cada herramienta instrumenta un universo diferente y un promedio ocultaría, por ejemplo, una cobertura móvil insuficiente detrás de resultados altos del Gateway. La Tabla 1 resume la pirámide realmente implementada.

6.1. Backend: pruebas unitarias, integración y cobertura

Los cinco módulos emplean Maven, JUnit 5 y JaCoCo; Mockito se usa donde es necesario aislar puertos o clientes. Las suites cubren controladores, servicios, repositorios, validaciones y seguridad. En Reservas también se prueban las estrategias de disponibilidad y las transiciones State; en Auth se ejercitan JWT, rotación de sesión, bloqueo de acceso, recuperación de contraseña y auditoría. La Tabla 2 describe los controles verificables en el árbol actual. Los porcentajes y recuentos exactos pertenecen a cada ejecución y deben consultarse en sus reportes Surefire/JaCoCo de GitHub Actions; no se trasladan cifras de un SHA anterior como si describieran el HEAD actual. La ejecución de push del SHA funcional `e43967eda31410b7fef060cfc7903cea94efbe96` completó los cinco jobs Backend, incluido el control reforzado de Reservas, sin sustituir esos reportes por una cifra agregada entre servicios.

Superar un umbral en una ejecución no significa que cada ruta esté probada ni que el sistema sea inmune a fallos. La cobertura de líneas no sustituye la verificación de

Cuadro 1: Niveles de prueba implementados en la Entrega 4.

Nivel	Tecnología	Componente	Propósito y evidencia
Unitario	JUnit, Mockito; Vitest/RTL; JUnit, MockK y Robolectric	Backend, Web y Mobile	Reglas, servicios, ViewModels, repositorios simulados, sesión, rutas y estados de interfaz; reportes Surefire, Vitest y Gradle.
Integración	Spring Boot, Testcontainers, Flyway, Room	Backend y Mobile	Persistencia, migraciones, seguridad, cifrado, concurrencia y migración local sobre implementaciones reales.
Contrato	Pact JVM	Web, Mobile y servicios	Compatibilidad de solicitudes y respuestas HTTP; JSON Pact y verificación de proveedores.
E2E	Playwright; Compose/Espresso	Web y Mobile	Flujos visibles en Chromium, Firefox, WebKit y emulador Android API 29.
Carga	Locust	Gateway y Reservas	Autenticación real, consultas nominales y rampa; CSV y archivos crudos analizados en el capítulo ISO.

condiciones, concurrencia o contratos. Por ello se conservan niveles de integración y E2E en el pipeline, coherentes con la necesidad de mantener pruebas a diferentes escalas [10].

6.1.1. Persistencia real con Testcontainers

Las pruebas de integración no se reducen a declarar la dependencia. En `Auth`, `UsuarioAuthRepository` inicia un `CockroachContainer`, persiste y recupera usuarios y comprueba que las sesiones activas excluyan registros vencidos o revocados. En `Usuarios`, `CockroachFlywayIntegrationTest` verifica la aplicación de Flyway, tablas, restricciones, índices y lectura/escritura mediante el adaptador; por su parte, `CifradoDatosSensiblesIntegrationTest` comprueba que los datos queden cifrados en la base, sean descifrados por la aplicación y puedan buscarse por hash.

`CockroachFlywayIntegrationTests`, en `Reservas`, levanta un contenedor efímero, valida las tablas migradas y ejecuta repositorios reales. Incluye serialización del mutex de agenda, aprobaciones concurrentes, atomicidad, versionado e idempotencia de creación y aprobación. Cada prueba obtiene una base aislada y las credenciales se generan para la ejecución. La limitación es concreta: `academico-laboratorios-service` carece de una prueba Testcontainers equivalente, aunque dispone de pruebas unitarias de adaptadores y de un contexto Spring de seguridad. El pipeline añade además un job separado que valida el clúster CockroachDB de tres nodos de la Entrega 3; esta evidencia se distingue de los contenedores efímeros por servicio.

6.2. Aplicación web

Vitest y React Testing Library validan el comportamiento observable de la SPA. El árbol actual contiene 31 archivos de prueba Web, que incluyen autenticación, rutas pro-

Cuadro 2: Controles de pruebas y cobertura Backend configurados.

Servicio	Herramienta	Fuente de resultado	Criterio
auth-service	JUnit/Mockito/JaCoCo	Surefire y JaCoCo del job Backend	Gate $\geq 70\%$
usuarios-service	JUnit/Mockito/JaCoCo	Surefire y JaCoCo del job Backend	Gate $\geq 70\%$
academico-laboratorios-service	JUnit/Mockito/JaCoCo	Surefire y JaCoCo del job Backend	Gate $\geq 70\%$
reservas-solicitudes-service	JUnit/Mockito/JaCoCo	Surefire y JaCoCo del job Backend	Gate: líneas $\geq 80\%$ y ramas $\geq 48\%$
api-gateway	JUnit/Mockito/JaCoCo	Surefire y JaCoCo del job Backend	Gate $\geq 70\%$

tegidas, cliente HTTP, renovación ante 401, APIs de dominio, formularios, reservas, administración y flujos por rol. Vitest/V8 aplica en `vite.config.ts` un umbral de 70 % a statements, branches, functions y lines. Los valores exactos deben obtenerse del job y del reporte del SHA evaluado; no se trasladan cifras de un árbol anterior como evidencia actual.

El pipeline ejecuta `npm run test:coverage` y `npm run build`; por tanto, una promoción solo continúa si los umbrales configurados y el build se satisfacen. La configuración demuestra el criterio exigido, mientras el reporte de la ejecución demuestra su resultado para un SHA concreto.

6.2.1. Compatibilidad E2E con Playwright

Playwright mantiene escenarios funcionales de autenticación, rutas protegidas, reservas, revisión y preferencias de interfaz. Cada escenario se proyecta sobre Chromium, Firefox y WebKit con un solo worker y hasta dos reintentos en CI. Los tres motores aportan evidencia práctica frente a diferencias de Chrome, Firefox y Safari, pero no representan todos los navegadores, sistemas operativos ni dispositivos.

La configuración y la ejecución deben distinguirse. La evidencia utilizada es el job *Compose integration tests* del SHA funcional `e43967eda31410b7fef060cfc7903cea94efbe96`, que terminó correctamente, instala los tres motores, espera healthchecks, realiza login y una consulta protegida, y luego ejecuta `npm run test:e2e`. El reporter es de consola; las trazas se habilitan en el primer reintento, pero el workflow no publica actualmente un artefacto Playwright dedicado.

6.3. Aplicación móvil

La suite local combina JUnit 4, MockK, `coroutines-test`, MockWebServer, Robolectric y utilidades de Compose. Sus pruebas ejercitan ViewModels, sesión y refresh, almacenamiento cifrado, clasificación de errores, repositorios remotos y locales, caché, reservas, incidentes, disponibilidad y estados QR. La última medición numérica conservada en la evidencia documental registra 45,69 % de líneas. El SHA funcional `e43967eda31410b7fef060cfc7903cea94efbe96` completó los jobs *Test mobile* y *Test mobile instrumented*, pero ese resultado verde no convierte la cifra histórica en una cobertura actual distinta ni permite afirmar 70 % sin

consultar el artefacto JaCoCo de la ejecución.

El resultado documentado es parcial: existe una base amplia de casos, pero la última evidencia versionada no alcanza 70 % de líneas. Compose, pantallas, navegación y wiring aportan una parte importante al denominador; aun así, esto no justifica ignorar el déficit. La configuración solo excluye artefactos Android generados habituales (por ejemplo, R, BuildConfig y manifest), no clases del negocio ni Composables para elevar artificialmente la cifra. Por ello la calidad móvil no se presenta como cumplimiento total.

6.3.1. Pruebas instrumentadas y APK

El directorio `androidTest` contiene cinco archivos de prueba; el número de casos ejecutados se obtiene del reporte instrumentado de cada SHA. Comprueba un flujo Compose de listado a detalle de reserva, DAO y actualización de caché, migración Room de versión 1 a 2, creación/listado de incidentes y tres estados del flujo QR. El job *Test mobile instrumented* usa un emulador Pixel 2, Android API 29, imagen `google_apis x86_64`, desactiva animaciones y ejecuta `connectedDebugAndroidTest`. Sus resultados se publican incluso ante fallo.

Este job depende de las pruebas locales y es gate del APK. A su vez, *Build mobile APK* genera `app-debug.apk` solo después de unitarias e instrumentadas. El proyecto no configura `signingConfig`, `keystore` ni `assembleRelease` en CI; por tanto, el artifact actual es exclusivamente de depuración y no se presenta como release firmado. No se encontró en el repositorio una rúbrica versionada que obligue a distribuir un release firmado, por lo que no se añadieron secretos ni complejidad de firma sin ese requisito. No se ejecutó un emulador manualmente durante la redacción; se usa la ejecución automatizada de CI como evidencia.

6.4. Contratos HTTP con Pact

Pact expresa el flujo consumidor → contrato → verificación del proveedor. El árbol contiene contratos consumidores para Web, Mobile, Auth, Usuarios, Académico y Reservas, además de verificadores de proveedor en Académico, Reservas y Usuarios. El número aprobado pertenece al reporte del job de cada SHA y no se fija aquí como una cifra inmutable. Los contratos de Reservas representan listar, consultar detalle, cancelar y proponer una alternativa; incluyen método, ruta, encabezado Bearer y forma del cuerpo. Los proveedores `reservas-solicitudes-service` y `usuarios-service` disponen de verificadores JUnit que preparan los estados requeridos y contrastan el JSON generado durante `mvn verify`.

CI regenera los Pacts antes de probar esos proveedores y publica los archivos JSON asociados al SHA. No existe Pact Broker: no hay matriz central de versiones, historial ni verificación automática de *can-i-deploy*. La compatibilidad se demuestra dentro de cada ejecución, pero su evolución entre despliegues exige consultar artefactos individuales.

6.5. Carga nominal y rampa

`tests/load/locustfile.py` autentica un usuario real contra el Gateway, conserva el JWT y genera, con pesos distintos, listados y detalles de reservas. Valida códigos y estructura mínima para no contabilizar como éxito una respuesta semánticamente inválida. `ramp_locustfile.py` incorpora un `LoadTestShape` que asciende linealmente de 0 a 200 usuarios durante diez minutos. CI solo ejecuta un smoke controlado de dos usuarios

durante diez segundos; la prueba nominal y la rampa experimental cuentan con evidencia real separada. Sus estadísticos y limitaciones se analizan en el capítulo de observabilidad e ISO 25010; no se reinterpretan en este capítulo.

6.6. Integración y entrega continuas

El workflow `ci-cd.yml` responde tanto a `pull_request` como a `push`. Un PR ejecuta validaciones, pero `build-images` está condicionado a `push` y, por tanto, no publica ni despliega. Los gates incluyen Checkstyle/ESLint; cinco matrices Maven con JaCoCo y verificación Pact; Vitest, cobertura y build Web; pruebas Mobile, lint, cobertura, emulador y APK; Testcontainers y clúster CockroachDB; Compose, Playwright y smoke Locust.

`build-images` depende explícitamente de `lint`, Backend, Web, Mobile local, Mobile instrumentado, APK, CockroachDB e Integración. Solo después construye las cinco imágenes de servicio y la Web. Esta estructura convierte las pruebas en barreras de promoción, una aplicación concreta del flujo automatizado DevSecOps descrito por NIST [11].

6.6.1. Artefactos y trazabilidad por SHA

Las imágenes se publican en GHCR con el SHA Git completo, nunca con `latest`. `docker-compose.prod.yml` exige `IMAGE_TAG` y lo aplica a las seis imágenes, estableciendo la cadena `commit` → ejecución CI → imagen → despliegue. Los demás artefactos publicados son los contratos Pact, `dist.tar.gz` de Web, el informe JaCoCo Mobile, los resultados instrumentados y el APK debug. No se publican actualmente los reportes JaCoCo Backend, la cobertura Web ni un reporte Playwright; permanecen en los logs o en el espacio temporal del runner.

6.6.2. Despliegue controlado

Tras publicar las imágenes, `call-deploy` invoca `cd.yml`. En un `push` a `feature/entrega-4` solo se activa cuando `CD_FEATURE_ENABLED=true`; en `main` requiere `CD_ENABLED=true`. El job verifica la identidad SSH configurada sin exponer claves, valida el fingerprint del servidor y se conecta mediante una Action fijada a `commit`. En la VM obtiene exactamente el SHA aprobado, adopta un checkout separado, inicia sesión en GHCR y entrega el mismo `IMAGE_TAG` a `deploy-vm.sh`.

El script valida la sintaxis Compose, descarga imágenes, ejecuta `up -d --remove-orphans` y espera hasta cinco minutos a que el healthcheck del Gateway responda UP. Un estado no saludable hace fallar el job y muestra diagnóstico, pero no restaura automáticamente la versión anterior. Tampoco hay estrategia blue/green, canary o garantía de zero-downtime; la recuperación es manual. La automatización reduce variación operacional, pero conserva estos límites, coherentes con la necesidad de tratar el despliegue como un proceso repetible y verificable [12].

Como aprendizaje operativo, la autenticación SSH falló una vez porque el proveedor cloud regeneró `authorized_keys` y retiró una entrada manual. La clave pública se registró mediante el mecanismo persistente de metadatos de la instancia. No fue un defecto del pipeline ni del código y no se reproducen IP, claves ni fingerprints en el informe.

Cuadro 3: Trazabilidad resumida de calidad, prueba y CI.

Requisito	Prueba	Job CI	Resultado/evidencia
Web $\geq 70\%$	Vitest y V8	Test web	Cuatro umbrales configurados en 70%; resultado por SHA en CI.
Calidad Mobile	JUnit/Robolectric, JaCoCo y emulador	Test mobile; Test mobile instrumented	Medición documental: líneas 45,69%; ambos jobs verdes en e43967e; cobertura numérica actual sujeta al artefacto JaCoCo.
Contratos compatibles	Pact consumer/provider	Matrices Backend	Consumidores y verificadores versionados; resultado por SHA en CI; sin Broker.
Persistencia CockroachDB	Testcontainers/Flyway	Matrices Backend; CockroachDB E3 cluster tests	Auth, Usuarios y Reservas verificados; falta equivalente en Académico.
Compatibilidad Web	Playwright	Compose integration tests	Escenarios versionados en tres motores; ejecución por SHA en CI.
Promoción trazable	Gates, GHCR y healthcheck	Build and publish image; Call production VM deployment	Imágenes y despliegue ligados al SHA completo.

6.7. Trazabilidad de requisitos de calidad

Finalmente, varias limitaciones de automatización permanecen abiertas. Algunas Actions oficiales usan tags mayores (por ejemplo, @v4 o @v6) en vez de commits inmutables; el lint Web usa Node 20 mientras pruebas e integración fijan Node 22.22.2; no hay rollback; y faltan artefactos persistentes para ciertas coberturas y Playwright. Estas restricciones no anulan los gates, pero delimitan la reproducibilidad y el endurecimiento de la cadena de suministro que todavía puede mejorarse.

7. Observabilidad y evaluación ISO/IEC 25010

La Entrega 4 incorpora observabilidad como una capacidad transversal del sistema distribuido y no como sustituto de las pruebas. Métricas, trazas y logs responden preguntas distintas: las métricas resumen comportamiento agregado, las trazas reconstruyen el recorrido de una solicitud y los logs conservan eventos discretos con contexto. Esta separación coincide con las señales definidas por OpenTelemetry [13]; combinarlas mejora el diagnóstico, pero la sola presencia de herramientas no certifica calidad ni garantiza correlación perfecta.

7.1. Arquitectura de observabilidad

Los cinco servicios Spring exponen Actuator y Prometheus, y configuran instrumentación y exportación OTLP. El flujo implementado es el siguiente:

```
servicios -- metricas Prometheus -----> Prometheus --\
          \-- trazas OTLP --> Collector --> Tempo -----> Grafana
            \- logs OTLP -----> Collector --> Loki -----/
cAdvisor -- recursos de contenedores -----> Prometheus ---/
```

Prometheus consulta cada quince segundos los cinco endpoints `/actuator/prometheus`, `cAdvisor`, el Collector y los tres nodos CockroachDB. Grafana dispone de fuentes provisionadas para Prometheus, Tempo y Loki. Tempo conserva trazas localmente durante 24 horas; Loki almacena logs, y cAdvisor aporta CPU y memoria de contenedores. El Collector recibe OTLP por gRPC 4317 y HTTP 4318, aplica el procesador `batch` y separa tres pipelines: trazas hacia Tempo, métricas hacia un exporter Prometheus en 8889 y logs hacia Loki. OpenTelemetry funciona aquí como estándar, instrumentación y canal de transporte; los backends y Grafana proporcionan almacenamiento y consulta, pues OpenTelemetry no es por sí mismo un APM completo.

7.1.1. Métricas Prometheus

Micrometer produce la familia `http_server_requests_seconds`: el contador permite calcular volumen y tasa por estado, y los buckets habilitan percentiles con `histogram_quantile`. En Reservas una regla de grabación crea el alias `http_request_duration_seconds` exigido por el protocolo a partir de la familia estándar. Usuarios y Académico agregan además `http_requests_total` con ruta, método y estado; Auth registra éxitos, fallos, refresh y logout; Reservas publica eventos de negocio de baja cardinalidad después del commit transaccional. Las series de CockroachDB y cAdvisor complementan la vista con estado de nodos, rangos no disponibles y consumo de recursos.

La tasa se obtiene con `rate` sobre contadores y los 5xx mediante el filtro `status=~"5.."`. Para latencia, los buckets se agregan por `le` antes de calcular p50, p95 o p99. Un percentil de histograma es una estimación dependiente de los buckets y de la ventana elegida; no equivale necesariamente al percentil calculado por el cliente de carga.

7.1.2. Trazas y logs estructurados

La configuración de cada servicio asigna `OTEL_SERVICE_NAME`, exporta trazas y logs por OTLP y habilita muestreo de trazas. Los appenders Logback se instalan sobre el SDK autoconfigurado y el formato de consola es JSON tipo Logstash. La configuración añade el servicio y renombra el contexto activo como `trace_id` y `span_id`; el encoder incorpora también timestamp, nivel, logger y mensaje. Tempo puede enlazar una traza con Loki usando ambos identificadores, aunque un log emitido fuera de un span no necesariamente los contiene. Por esa razón se habla de correlación disponible, no universal.

Los registros no deben transportar credenciales. En particular, el callback de FCM informa que recibió un token nuevo, pero no escribe su valor. Esta decisión preserva utilidad operativa sin convertir el sistema de logs en un canal de exposición. Los eventos estructurados y ricos en contexto facilitan consultas y diagnóstico de sistemas distribuidos [14], pero su calidad depende de atributos consistentes y de la propagación efectiva.

7.2. Dashboard consolidado

`ops/grafana/pfc-dashboard.json` contiene exactamente seis paneles. Las consultas PromQL son sintácticamente coherentes con las familias configuradas; la Tabla 4 distingue lo que mide cada una y evita atribuir al panel una semántica que no posee.

Cuadro 4: Paneles reales del dashboard consolidado.

Panel	Fuente	Métrica/consulta	Propósito
Request rate	Prometheus	<code>rate(http_server_requests_seconds_count)</code>	Contar peticiones por job en cinco minutos.
Latencia p50/p95/p99	Prometheus	<code>histogram_quantile</code> sobre buckets HTTP	Observar distribución y cola de latencia.
Errores HTTP 5xx	Prometheus	Tasa del contador HTTP con estado 5xx	Detectar fallos de servidor por job.
CockroachDB	Prometheus	<code>liveness_livenodes</code> ; <code>ranges_unavailable</code>	Vigilar nodos vivos y rangos no disponibles.
Recursos de contenedores	Prometheus/cAdvisor	Uso de CPU y memoria por nombre de contenedor	Identificar presión de recursos.
Latencia E2E móvil (proxy)	Prometheus	p95 del Gateway para URI de reservas, solicitudes, incidentes o QR	Aproximar endpoints asociados a Mobile; no mide dispositivo, red completa ni renderizado.

El sexto panel constituye la brecha principal: su nombre reconoce que es un proxy y su punto de medición sigue siendo el Gateway. Además, algunas URI del filtro no tienen backend remoto actual, como incidentes. Por tanto, el dashboard cubre seis finalidades solicitadas, pero la latencia Mobile no es E2E real y el criterio D6.2 permanece parcial. No se incorporaron capturas, porque no existe una imagen versionada actual del dashboard que aporte evidencia adicional al JSON reproducible.

7.3. Marco y protocolo ISO/IEC 25010

La evaluación usa el modelo de calidad de producto ISO/IEC 25010:2023 [3], pero no pretende cubrir todas sus características. Se seleccionaron eficiencia de desempeño, fiabilidad, seguridad, mantenibilidad y compatibilidad. La disponibilidad se trata como parte de fiabilidad y exige observación temporal propia; no se infiere de una tasa puntual de errores.

El protocolo nominal define diez repeticiones equivalentes con 50 usuarios. La eficiencia dura cinco minutos y exige p95 menor que 500 ms. La fiabilidad preveía una hora por repetición y menos de 1 % de HTTP 5xx. La regla se fijó antes de observar resultados: r1 y r10 se conservan, mientras r2–r9 forman las ocho muestras centrales. Esto trata de forma uniforme los bordes del lote y, sobre todo, impide elegir exclusiones después de conocer los valores; no se eliminaron ejecuciones desfavorables por conveniencia.

Para cada métrica central se usa $n = 8$, desviación estándar muestral con siete grados de libertad e intervalo bilateral $\bar{x} \pm t_{0,975;7}s/\sqrt{8}$, con $t = 2,364624251$. La decisión

principal es conservadora: el límite superior del IC95 debe ser menor que el umbral. Esta estructura y la conservación de resultados fallidos son coherentes con un proceso de prueba documentado y repetible [9].

7.3.1. Proveniencia e integridad

La evidencia se ejecutó en `servidor-proyectos` sobre el SHA `a47f0441f644bea5f52944b7a11216f3` con Locust 2.31.6 y Python 3.11.10. El protocolo está en `../..../experimentos/protocolo-e4.md`; `../..../experimentos/resultados/iso25010.csv` contiene resultados procesados y `../..../experimentos/resultados/raw` conserva CSV canónicos, fallos, excepciones, logs, metadata, consultas y respuestas Prometheus, salud y estadísticas de contenedores. Los historiales completos y HTML voluminosos permanecen en la VM, circunstancia registrada en el resumen de evidencia.

El manifiesto `../..../experimentos/resultados/SHA256SUMS` enumera 195 archivos. La verificación durante esta redacción confirmó 195 coincidencias, sin archivos ausentes ni hashes distintos. SHA-256 permite detectar alteraciones respecto del manifiesto, pero no constituye una firma ni prueba autenticidad por sí solo. La procedencia se apoya adicionalmente en metadata, timestamps, SHA probado y el documento `../..../experimentos/resultados/RESUMEN-ISO25010-E4.md`.

7.4. Eficiencia nominal: 50 usuarios durante cinco minutos

Las diez repeticiones terminaron con código cero y registraron 0 respuestas HTTP 5xx. La Tabla 5 transcribe el CSV verificado; r1 y r10 se muestran porque forman parte de la evidencia, aunque no intervienen en el análisis central.

Cuadro 5: Diez repeticiones de eficiencia nominal medidas por Locust.

Rep.	Requests	HTTP 5xx	p95 (ms)	p99 (ms)	Uso
r1	7041	0	110	2200	Conservada
r2	7230	0	25	53	Central
r3	7151	0	130	1600	Central
r4	7212	0	25	89	Central
r5	7205	0	63	1400	Central
r6	7259	0	34	150	Central
r7	7214	0	69	570	Central
r8	7209	0	29	270	Central
r9	7161	0	85	860	Central
r10	7281	0	25	150	Conservada

Las ocho repeticiones centrales suman 57 641 solicitudes. Para p95, los valores 25, 130, 25, 63, 34, 69, 29 y 85 ms producen media 57,500000 ms, desviación muestral 37,132966 ms e IC95 [26,456064; 88,543936] ms. Como el límite superior 88,543936 ms es menor que 500 ms, la eficiencia **cumple** para este escenario nominal. La conclusión no se extrapola a cargas, duraciones o entornos distintos.

El p99 se analiza como indicador secundario. Los valores 53, 1600, 89, 1400, 150, 570, 270 y 860 ms producen media 624,000000 ms, desviación 606,585526 ms e IC95 [116,881810; 1131,118190] ms. El límite superior excede el umbral secundario de 750 ms usado por el analizador, por lo que ese indicador no cumple. Este umbral no es el criterio

principal p95 de la rúbrica: el resultado revela cola larga y alta variabilidad, pero no cambia retrospectivamente la decisión p95.

Las ocho ventanas centrales contienen 0 HTTP 5xx de 57 641 solicitudes, es decir, 0 %, frente al objetivo menor que 1 %. Esto demuestra ausencia de 5xx en el escenario observado; no demuestra disponibilidad anual, ausencia de otros errores ni fiabilidad absoluta.

7.4.1. Contraste con Prometheus

Las series Prometheus de r2–r9 contienen p95 de 10,004445; 30,067916; 9,949600; 21,021640; 13,281961; 22,143963; 11,358052 y 28,439426 ms. Su media es 18,283375 ms, la desviación 8,240924 ms y el IC95 [11,393791; 25,172960] ms; las diez ventanas registran cero 5xx. Locust mide extremo a extremo desde el generador, incluyendo red, Gateway y serialización. Prometheus mide el histograma interno de Reservas para su ventana. Son puntos de observación complementarios, no resultados intercambiables ni fuentes que deban forzarse a coincidir.

7.5. Rampa exploratoria de 0 a 200 usuarios

La rampa de diez minutos produjo 29,217 solicitudes, una respuesta HTTP 500 en `GET /api/v1/reservas` y una tasa exacta de 0,00342266 %. Locust registró p95 de 180 ms, p99 de 1100 ms y máximo de 14024,163663 ms. El proceso terminó con estado fallido porque se configuró para devolver error ante cualquier fallo, no porque la mayoría de solicitudes hubiera fallado.

La tasa 5xx permaneció por debajo de 1 % y el p95 fue moderado mientras la carga crecía, pero p99 y el máximo confirman una cola larga y picos aislados. Esta prueba complementa el estudio de capacidad progresiva; no pertenece al CSV central ni sustituye el escenario nominal ISO.

7.6. Fiabilidad y disponibilidad: alcance no concluido

El protocolo de fiabilidad preveía diez repeticiones independientes de una hora con 50 usuarios. No se ejecutó ninguna: **NO EJECUTADA** — 0/10. La campaña requiere al menos diez horas efectivas de observación continua, además de preparación, monitoreo, recolección y validación de evidencia; ese trabajo no se completó dentro de la ventana disponible de la entrega. Por integridad experimental no se reportan ejecuciones parciales, la prueba de cinco minutos ni la rampa como sustitutos. No se simulaban horas, no se sustituyeron datos ausentes por cero y el analizador informa 0/8 muestras centrales completas.

La tasa HTTP 5xx y la disponibilidad no son equivalentes. Para concluir que la disponibilidad es al menos 99,5 % se requiere una observación temporal válida y el protocolo completo. En consecuencia, fiabilidad temporal y disponibilidad son **no concluyentes**; el informe no afirma 99,5 %, 100 % ni aprobación de fiabilidad.

7.7. Experimento de arbitraje concurrente S0–S4

El experimento ARBITER comparó cinco comportamientos seleccionables sin alterar el camino productivo predeterminado: S0, escritura sin arbitraje usada como control negativo; S1, control optimista; S2, control pesimista; S3, coordinador de aplicación elegido mediante el algoritmo Bully y orden lógico de Lamport; y S4, transacción **SERIALIZABLE**

en CockroachDB respaldada por el consenso del clúster. S3 no utiliza Raft: su líder pertenece a la aplicación. En S4 no existe un coordinador Bully; la coordinación corresponde a la persistencia distribuida del clúster.

La campaña del SHA `e43967eda31410b7fef060cfc7903cea94efbe96` planificó 130 corridas. Se completaron 123, siete fallaron y ninguna fue clasificada como inválida. La regla prerregistrada excluye las repeticiones 1 y 10 de cada configuración, lo que produce 104 posiciones analíticas teóricas. Cinco posiciones centrales de Esc-1 también fallaron; por ello el conjunto realmente analizable contiene 99 corridas completadas. La Tabla 6 evita confundir esas cuatro cantidades.

Cuadro 6: Conteos auditados de la campaña ARBITER.

Escenario	Planificadas	Completadas	Fallidas	Analíticas válidas
Esc-1, nominal	10	3	7	3
Esc-2, contención media	50	50	0	40
Esc-3, contención máxima	50	50	0	40
Esc-4, caída controlada	20	20	0	16
Total	130	123	7	99

S0 exhibió dobles adjudicaciones en todas las muestras analíticas de contención: 8/8 en Esc-2 y 8/8 en Esc-3. Para S1–S4 no se observaron dobles adjudicaciones en las corridas ejecutadas; este resultado acotado no constituye una garantía de aislamiento absoluto. La Tabla 7 resume las 96 muestras analíticas de contención y caída, separadas de las tres muestras nominales válidas de Esc-1.

Cuadro 7: Resultado principal por estrategia en muestras analíticas.

Estrategia	n	Dobles adjudicaciones	Interpretación
S0	16	Observadas en 16/16	Control negativo efectivo
S1	16	No observadas	Resultado limitado a las corridas
S2	16	No observadas	Resultado limitado a las corridas
S3	24	No observadas	Incluye 8 corridas de caída
S4	24	No observadas	Incluye 8 corridas de caída

En S3, los tiempos de recuperación de las ocho muestras analíticas fueron 0, 0, 1, 0, 0, 0, 0 y 0 ms: media 0,125 ms, mediana 0 ms y rango 0–1 ms. En siete casos no se observó demora medible con la resolución milimétrica disponible; esto no equivale a recuperación instantánea. En S4, la recuperación observada por healthcheck presentó media 3252,11 ms, mediana 3171,85 ms, desviación estándar 2188,52 ms y rango 1116,75–5949,68 ms. Esta medición demuestra el retorno del servicio al estado saludable, no el tiempo interno de convergencia de Raft.

El oráculo evaluó 96 muestras analíticas. I1 (ausencia de doble adjudicación) produjo 80 PASS y 16 FAIL, todos estos últimos en S0; I2 (exactamente un adjudicatario) produjo 96 PASS; e I3 (equipo fuera de mantenimiento), 96 PASS. I4 (liberación exactamente una vez tras cancelar) e I5 (acceso solo con reserva vigente) quedaron ambos en 96 NOT_OBSERVED; no se interpretan como aprobados.

Para la tasa de doble adjudicación, las comparaciones prerregistradas entre S0 como grupo A y cada estrategia S1–S4 como grupo B, tanto en Esc-2 como en Esc-3, obtuvieron $U = 64$, $p = 0,0007775$ y $A_{12} = 1,0$, con ocho muestras por grupo. A_{12} expresa la probabilidad de que A produzca un valor mayor que B: en esta métrica, un valor mayor es desfavorable, de modo que 1,0 representa el peor comportamiento de S0 y no su

superioridad. Las comparaciones de latencia mostraron diferencias entre S0 y S3 en Esc-2 ($U = 4$, $p = 0,00328$, $A_{12} = 0,0625$) y entre S0 y S1–S3 en Esc-3 ($p \leq 0,00114$ y $A_{12} \leq 0,015625$); S0 frente a S4 no mostró diferencia estadística en Esc-2 ($p = 0,7527$) ni Esc-3 ($p = 0,5286$). Estos contrastes no compensan la inseguridad observada en S0.

El índice de Jain fue 1,0000, pero no discrimina equidad en este diseño: cada corrida usa una única franja y un único equipo y, en S1–S4, normalmente existe un solo adjudicatario. Se trata de un resultado degenerado, no de evidencia de superioridad. Los rechazos innecesarios medidos fueron cero porque todas las solicitudes contenciosas compartieron equipo y franja con una adjudicación confirmada. Además, algunas corridas marcadas como completadas contienen `HTTP_ERROR`; siete corridas de Esc-1 fallaron ante respuestas `HTTP 500` y este escenario aporta solo $n = 3$ muestras válidas; I4 e I5 no fueron observados; S3 está limitado por la resolución temporal; y S4 mide healthcheck, no consenso directamente. La evidencia solo permite afirmar que no se observaron modificaciones en las tablas productivas que fueron verificadas, no impacto cero absoluto sobre toda la producción.

7.8. Evaluación ISO/IEC 25010 seleccionada

Observabilidad apoya la eficiencia, la tasa de fallos y el diagnóstico, pero no certifica ISO 25010 por sí sola. La Tabla 8 integra solo características con evidencia disponible.

En seguridad, la matriz actual inventaría 183 operaciones de controlador y 15 Actuator. Se identifican 160 operaciones externas con JWT, 145 con RBAC explícito, 18 internas con API key y 63 con control de ámbito; estos conjuntos no son equivalentes ni justifican declarar cobertura total frente a todas las amenazas. Persisten limitaciones de scope documentadas en Agenda e Incidentes. En mantenibilidad existen gates de cobertura para Backend y Web. El push del SHA funcional `e43967eda31410b7fef060cfc7903cea94efbe96` completó los cinco jobs Maven y el job Web; Reservas exige además 80 % de líneas y 48 % de ramas. La última evidencia Web documentada registra 86,55 % de líneas y 70,33 % de ramas. Mobile conserva como referencia documental 45,69 % de líneas, bajo 70 %; el job verde no autoriza a reemplazar esa cifra sin leer su reporte. En compatibilidad, el mismo SHA completó Compose/Playwright y la suite instrumentada Android API 29. Android declara API 26 o superior, pero una ejecución en API 29 y tres motores Web no implican compatibilidad universal.

7.9. Mantenibilidad: evidencia y alcance

La **modularidad** se sustenta en cinco microservicios con propiedad de datos diferenciada y en clientes Web y Android separados. Dentro de los servicios, controladores, casos de uso, puertos y adaptadores reducen el impacto de sustituir persistencia o clientes HTTP. Esta separación no es absoluta: el dominio Académico todavía expone tipos de paginación de Spring, limitación descrita en la Sección 3.

La **analizabilidad** se apoya en Checkstyle, ESLint, logs estructurados, métricas, trazas y reportes Surefire, JaCoCo y V8. La arquitectura registra decisiones mediante ADR y emplea Repository, Strategy, State, Observer, Facade, Factory y Singleton en puntos concretos; estos patrones aportan vocabulario y puntos de variación, pero su sola presencia no prueba bajo acoplamiento.

La **modificabilidad** se evidencia mediante contratos Pact y gates por servicio: un cambio se valida antes de construir imágenes ligadas al SHA. La **testabilidad** cuenta con

Cuadro 8: Resultado de las características ISO/IEC 25010 seleccionadas.

Característica	Métrica	Objetivo	Evidencia/resultado	Estado
Eficiencia	p95 nominal, 50 usuarios	IC95 superior < 500 ms	88,543936 ms	CUMPLE
Eficiencia secundaria	p99 nominal	IC95 superior < 750 ms	Media 624,00 ms; SD 606,59 ms; IC95 [116,88; 1131,12] ms	NO CUMPLE
Fiabilidad acotada	HTTP 5xx nominal	< 1 %	0/57 641, 0 % en r2–r9	CUMPLE en escenario
Disponibilidad	Observación temporal	$\geq 99,5$ %	Fiabilidad 0/10; sin horas válidas	NO CONCLUYENTE
Seguridad	JWT, RBAC y matriz de endpoints	Acceso autenticado y autorizado	198 operaciones: 160 JWT, 145 con RBAC, 18 internas con API key y 63 con scope explícito	PARCIAL
Mantenibilidad	Modularidad, pruebas, cobertura y complejidad	Gates por componente; complejidad < 10 donde fue medida	Capas, patrones y CI; Backend/Web verdes en e43967e ; complejidad medida solo en Usuarios y Mobile con 45,69 % histórico	PARCIAL
Compatibilidad	Motores Web y Android	Chromium, Firefox, WebKit; API 26+	Compose/Playwright y Android API 29 verdes en el push e43967e ; alcance acotado a esos entornos	PARCIAL

unitarias, integración Spring/Room, Testcontainers, pruebas de seguridad, Pact, Vitest, Playwright e instrumentadas Android. Los umbrales son verificables en configuración: 70 % de líneas para Auth, Usuarios, Académico y Gateway; 80 % de líneas y 48 % de ramas para Reservas; y 70 % para statements, ramas, funciones y líneas Web.

La conclusión permanece **PARCIAL**. No existe una medición de complejidad global homogénea para todos los componentes, la medición versionada de complejidad está acotada a Usuarios, faltan artefactos persistentes de cobertura Backend/Web y la cifra Mobile conservada está bajo 70 %. Por ello no se deriva un cumplimiento total a partir de que los gates del SHA funcional hayan sido verdes.

7.10. Análisis crítico

Las fortalezas principales son una p95 nominal con amplio margen respecto de 500 ms, cero 5xx en las muestras centrales, instrumentación de las tres señales, evidencia raw trazable, pruebas multicapa y gates CI/CD ligados al SHA. La arquitectura permite relacionar un cambio con pruebas, imagen y despliegue, y el manifiesto hace detectables alteraciones en el conjunto experimental.

Las limitaciones impiden una conclusión total. El p99 presenta gran dispersión y la rampa contiene un máximo superior a 14 segundos; Mobile permanece bajo 70 % de co-

bertura; fiabilidad 10 por 1 hora no se inició y la disponibilidad no fue demostrada. A ello se suman el panel Mobile proxy, falta de rollback automático, ausencia de Pact Broker, Testcontainers no uniforme y artefactos de calidad no persistidos para todos los componentes. Estas restricciones se documentan como amenazas a la generalización, no se ocultan mediante promedios o extrapolaciones.

7.11. Reflexión ética

SCLI trata perfiles de estudiantes y docentes, asignaciones institucionales, reservas, incidentes y registros de asistencia. Aunque el sistema no está integrado con la matrícula académica oficial ni con fuentes externas de la universidad, estos datos internos permiten inferir presencia, horarios y actividad. Por ello deben recopilarse únicamente los atributos necesarios para operar el laboratorio, limitarse su conservación al propósito declarado y evitar que observabilidad o logs repliquen credenciales, tokens o información personal innecesaria.

La trazabilidad de aprobaciones y asistencia favorece la rendición de cuentas, pero también puede convertirse en vigilancia excesiva si se usa para finalidades distintas de las informadas. El control por roles reduce exposición, sin eliminar la obligación de revisar periódicamente permisos y accesos. Las métricas deben agregarse cuando sea posible y no utilizarse para elaborar perfiles personales de productividad. Estas decisiones siguen los principios de privacidad, prevención del daño, honestidad y responsabilidad profesional del Código de Ética de la ACM [15].

Las decisiones académicas o disciplinarias no deben automatizarse a partir de una ausencia, alerta o métrica aislada. SCLI aporta evidencia operacional; una persona responsable debe verificar contexto, errores de identificación, indisponibilidad técnica y reglas institucionales antes de actuar. El equipo mantiene responsabilidad sobre diseño, validación y comunicación de limitaciones, incluida la ausencia de integración con registros académicos oficiales.

La responsabilidad profesional también alcanza la seguridad del producto y la veracidad de su evaluación. Los permisos deben aplicarse en Backend y no solo ocultar controles en las interfaces; los incidentes y trazas deben conservarse sin exponer datos personales; y ninguna automatización debe desplazar el juicio humano en decisiones que afecten a estudiantes o docentes. En coherencia con el Código de Ética y Práctica Profesional de Ingeniería de Software, aprobado conjuntamente por ACM e IEEE Computer Society [16], el equipo debe comunicar tanto los resultados favorables como los fallos y límites. Por ello se mantienen las siete corridas fallidas de ARBITER, los invariantes no observados y la fiabilidad 0/10, en lugar de convertir ausencia de evidencia en cumplimiento.

7.12. Declaración de uso de inteligencia artificial generativa

Durante el desarrollo y la redacción se empleó inteligencia artificial generativa como herramienta de apoyo. El código y el texto resultantes fueron revisados por integrantes del equipo y contrastados con el repositorio, las pruebas y la evidencia disponible. Una salida generada por IA no se considera por sí sola evidencia experimental: los resultados reportados proceden de los archivos de protocolo, datos crudos, resúmenes y verificaciones versionadas. La selección técnica, la interpretación de resultados y la responsabilidad final corresponden al equipo FUVV; no se atribuye a la herramienta autoría ni decisión autónoma.

7.13. Amenazas a la validez

7.13.1. Validez interna

Amenaza: calentamiento de JVM, cachés y recursos compartidos entre contenedores pueden alterar la latencia. **Efecto:** diferencias entre repeticiones que no provienen del cambio evaluado. **Mitigación aplicada:** protocolo previo, diez repeticiones nominales, conservación de r1 y r10 y uso uniforme de r2–r9 para el análisis central. **Riesgo residual:** no se aislaron por completo CPU, red, cachés ni pausas del runtime en el host.

Amenaza: la fiabilidad temporal quedó sin ejecutar. **Efecto:** una tasa puntual de 5xx podría interpretarse erróneamente como disponibilidad. **Mitigación aplicada:** se conserva explícitamente el resultado 0/10 y no se calcula disponibilidad. **Riesgo residual:** la fiabilidad de diez ejecuciones de una hora sigue siendo desconocida.

7.13.2. Validez externa

Amenaza: la evidencia procede de una VM y un conjunto de datos y carga controlados, no de una instalación distribuida representativa de todas las condiciones institucionales. **Efecto:** p95, p99 y tasa de errores pueden cambiar con otra red, hardware, volumen o distribución de usuarios. **Mitigación aplicada:** se documentan versión, SHA, herramientas, parámetros y datos crudos para repetir el escenario. **Riesgo residual:** los resultados solo se generalizan al escenario medido.

Amenaza: el número de escenarios de carga y compatibilidad es limitado. **Efecto:** pueden quedar combinaciones de navegador, dispositivo o contención sin observar. **Mitigación aplicada:** se usan tres motores Web, un emulador Android y una rampa exploratoria además del caso nominal. **Riesgo residual:** dispositivos reales y redes adversas permanecen sin cubrir; aunque ARBITER S0–S4 fue ejecutado, Esc-1 conservó solo tres muestras analíticas válidas y el dataset experimental no representa toda la demanda de un campus real.

7.13.3. Validez de constructo

Amenaza: Locust y Prometheus miden latencia en puntos diferentes, y el panel Mobile utiliza un proxy del Gateway. **Efecto:** tratar las métricas como equivalentes sobreestimaría la experiencia E2E o escondería latencia de red y renderizado. **Mitigación aplicada:** se reportan por separado y se explica el punto de medición. **Riesgo residual:** no existe medición E2E real desde el dispositivo.

Amenaza: cobertura, número de pruebas y configuración del pipeline no equivalen por sí solos a calidad completa. **Efecto:** declarar cumplimiento por conteo puede omitir seguridad, concurrencia o comportamiento semántico. **Mitigación aplicada:** se combinan pruebas unitarias, contratos, integración, E2E y evidencia experimental, y las cifras variables se remiten al reporte del SHA. **Riesgo residual:** Seguridad, Compatibilidad y Mantenibilidad permanecen parciales donde falta evidencia integral actual.

7.14. Trazabilidad final de la Entrega 4

7.15. Conclusiones

La Entrega 4 consolidó un sistema distribuido con cinco aplicaciones Backend, clientes Web y Android, persistencia CockroachDB y controles de seguridad integrados. La

Cuadro 9: Matriz final de trazabilidad respecto de la rúbrica.

Req.	Implementación	Evidencia	Sección	Estado
D1.1	Capas y puertos	Servicios; dependencia Spring Page/Pageable en dominio Académico	Arquitectura/SOLID	PARCIAL
D1.2	Siete patrones reales	ADR-005 y clases Repository, Factory, Strategy, State, Observer, Facade, Singleton	Patrones	CUMPLE
D2.1	SPA Web funcional y logout remoto	React, rutas, reservas, JWT y revocación	Aplicación Web	CUMPLE
D2.2	Calidad Web	Umbral V8 al 70%; Playwright en tres motores	Pruebas Web	Resultado por SHA en CI
D3.1	Cliente Android	Compose/MVVM, Room, QR; incidentes locales y FCM no E2E	Aplicación Mobile	PARCIAL
D3.2	Calidad Mobile	Pruebas unitarias e instrumentadas; reporte del HEAD con líneas 45,69 %	Pruebas Mobile	PARCIAL
D4.1	Contratos Pact	Consumidores y verificadores versionados; resultado por SHA	Pact	EVIDENCIADO EN CI
D4.2	Integración CockroachDB	Testcontainers, Flyway, cifrado e idempotencia	Testcontainers	CUMPLE
D5.1	Pirámide de pruebas	Backend, Web, Mobile, integración y E2E verdes en e43967e; Mobile conserva medición documental bajo 70 %	Estrategia de pruebas	PARCIAL
D5.2	CI/CD con gates y SHA	Workflows, GHCR, APK, deployment; sin rollback y Actions por tag	CI/CD	PARCIAL
D6.1	Métricas, trazas y logs	Prometheus, OTel, Tempo, Loki, cAdvisor y Grafana	Observabilidad	CUMPLE
D6.2	Dashboard de seis paneles	JSON canónico; latencia Mobile solo proxy de Gateway	Dashboard	PARCIAL
D7.1	Experimento ISO 25010	Eficiencia completa; hashes 195/195; fiabilidad 0/10	ISO/experimentos	PARCIAL
D7.2	Experimento ARBITER	130 planificadas; 123 completadas; 99 muestras analíticas válidas	ARBITER/oráculo	PARCIAL

arquitectura por capas, los patrones aplicados y los contratos HTTP permiten separar responsabilidades, aunque permanece deuda puntual en el dominio Académico y en capacidades Mobile sin backend remoto.

La calidad cuenta con evidencia reproducible: pruebas unitarias, integración real con CockroachDB, Pact, Playwright, Android instrumentado, Locust y un pipeline que condiciona imágenes y despliegue. Backend y Web tienen gates de cobertura configurados; la evidencia Mobile del HEAD permanece bajo 70 %, por lo que no se afirma cumplimiento global ni se atribuyen al HEAD cifras de un reporte anterior.

La eficiencia nominal cumple el criterio p95, y las muestras centrales no registraron HTTP 5xx. Sin embargo, p99 y la rampa muestran colas largas. La fase de fiabilidad no se ejecutó, por lo que no existe base para afirmar disponibilidad de 99,5 %. La conclusión profesional es, por tanto, favorable para el escenario de eficiencia medido y parcial para la evaluación ISO completa.

El experimento ARBITER confirmó la utilidad del control negativo: S0 presentó dobles adjudicaciones en las 16 muestras analíticas de contención. No se observaron dobles adjudicaciones con S1–S4 en las corridas ejecutadas, sin que esto constituya una garantía universal. De 130 corridas planificadas se completaron 123 y quedaron 99 muestras analíticas válidas; I4 e I5 no fueron observados y las mediciones de recuperación conservan las limitaciones de resolución y healthcheck descritas en la Sección 7.7.

La observabilidad mejora la capacidad de diagnóstico mediante Prometheus, OpenTe-

lemetry, Tempo, Loki, cAdvisor y Grafana. Su dashboard satisface la estructura de seis paneles, pero debe sustituir el proxy Mobile por medición E2E real. En conjunto, la entrega está sustentada para informe y defensa siempre que las limitaciones experimentales y técnicas permanezcan explícitas.

7.16. Trabajo futuro

Las prioridades son elevar la cobertura Mobile con casos de ramas y UI de valor, completar las diez horas de fiabilidad bajo un protocolo autorizado y validar disponibilidad, investigar y reducir la variabilidad p99, y medir latencia E2E desde el dispositivo. También conviene automatizar rollback, incorporar Pact Broker con *can-i-deploy*, extender Testcontainers a Académico, persistir reportes Backend/Web/Playwright y fijar versiones inmutables y uniformes de las herramientas CI. Estas acciones cierran brechas existentes sin introducir tecnología ajena a los riesgos observados.

Referencias

- [1] L. Bass, P. Clements, and R. Kazman, *Software Architecture in Practice*. Addison-Wesley Professional, 4 ed., 2021.
- [2] D. Berardi, S. Giallorenzo, J. Mauro, A. Melis, F. Montesi, and M. Prandini, “Microservice security: A systematic literature review,” *PeerJ Computer Science*, vol. 7, p. e779, 2022.
- [3] ISO/IEC, “ISO/IEC 25010:2023 Systems and Software Engineering—Systems and Software Quality Requirements and Evaluation (SQuaRE)—Product Quality Model,” Tech. Rep. ISO/IEC 25010:2023, International Organization for Standardization, 2023.
- [4] I. K. Aksakalli, T. Çelik, A. B. Can, and B. Tekinerdogan, “Deployment and communication patterns in microservice architectures: A systematic literature review,” *Journal of Systems and Software*, vol. 180, p. 111014, 2021.
- [5] R. Chandramouli and Z. Butcher, “Building secure microservices-based applications using service-mesh architecture,” Tech. Rep. NIST SP 800-204A, National Institute of Standards and Technology, 2020.
- [6] T. Lodderstedt, J. Bradley, A. Labunets, and D. Fett, “Best current practice for oauth 2.0 security,” Tech. Rep. RFC 9700, Internet Engineering Task Force, 2025.
- [7] P. Späth, *Pro Android with Kotlin: Developing Modern Mobile Apps with Kotlin and Jetpack*. Apress, 2 ed., 2022.
- [8] Android Developers, “Build an offline-first app.” Documentación técnica oficial de Android. Consultado el 30 de agosto de 2026.
- [9] ISO/IEC/IEEE, “ISO/IEC/IEEE 29119-2:2021 Software and Systems Engineering—Software Testing—Part 2: Test Processes,” Tech. Rep. ISO/IEC/IEEE 29119-2:2021, International Organization for Standardization, 2021.

- [10] T. Winters, T. Manshreck, and H. Wright, *Software Engineering at Google: Lessons Learned from Programming Over Time*. O'Reilly Media, 2020.
- [11] R. Chandramouli, "Implementation of devsecops for a microservices-based application with service mesh," Tech. Rep. NIST SP 800-204C, National Institute of Standards and Technology, 2022.
- [12] D. Farley, *Modern Software Engineering: Doing What Works to Build Better Software Faster*. Addison-Wesley Professional, 2021.
- [13] OpenTelemetry Authors, "Signals." Documentación oficial de OpenTelemetry, 2026. Consultado el 30 de agosto de 2026.
- [14] C. Majors, L. Fong-Jones, and G. Miranda, *Observability Engineering: Achieving Production Excellence*. O'Reilly Media, 2022.
- [15] Association for Computing Machinery, "ACM Code of Ethics and Professional Conduct." Código de ética profesional de la ACM, 2018. Consultado el 3 de septiembre de 2026.
- [16] ACM/IEEE-CS Joint Task Force on Software Engineering Ethics and Professional Practices, "Software Engineering Code of Ethics and Professional Practice, Version 5.2." Código profesional aprobado conjuntamente por ACM e IEEE Computer Society, 1999. Consultado el 4 de septiembre de 2026.